

Havelka Clubs

By

Marina Erceg, Joey Endres, Isobel Bartels, and David Taylor



ISA 245 B Final Project

haclubs/ha4clubs

Submitted to Professor Douglas Havelka

Miami University

May 14, 2025

01 Project Overview

In place of a final exam our team was tasked with compiling the skills weve learned this semester into a database project. In creating a database for our golf course, Havelka Clubs, we aim to achieve the following;

1. Analyze the information needs of a business and represent these needs with an entity-relationship diagram.
2. Design and implement a complex database.
3. Translate entity-relationship diagrams into tables for a relational DBMS using SQL DDL.
4. Write simple and advanced data retrieval and maintenance commands using SQL.
5. Present technical material in a professional, business manner

The Havelka Clubs database supports the operational needs of a public golf course, including tee time reservations, cart assignments, and point-of-sale (POS) transactions for clubhouse purchases.

The Following are the Assumptions and Business Rules

1. Each round includes the use of a cart.
2. Each cart holds 1-2 people, with a group of 2 or less having 1 cart and a group of 3 or 4 having 2 carts.
3. All food and beverage purchases occur at the clubhouse.
4. Only unbooked (empty) tee times can be reserved; random or pre-existing group joins are not allowed.

5. Each group or customer makes a single consolidated payment per booking.
6. Customers can exist in the system without any purchases, as their information must be entered before booking a tee time.
7. The Product table stores the purchase cost, while the SaleDetail table stores the selling price.

02 Data Model & Design

The ER diagram (*Figure 1.0*) represents the core business functions of Havelka Clubs, including customer management, tee time bookings, cart assignments, and sales transactions for products.

Entities and their key relationships:

Customer: Stores customer information, required before any transaction or tee time booking.

SalesHeader & Sales_Details: Represent the structure of sales transactions, supporting multi-item purchases.

Product: Contains product information such as type (food, drink, merchandise) and base pricing.

TeeTimes: Tracks each booking, start time, number of players, and round type.

CartAssignment & Carts: Ensures that each round has a cart assigned; carts include VIN and the date purchased data.

Relationships and Business Rules:

Customer → SalesHeader: A customer may have zero or more transactions.

SalesHeader → Sales_Details: A transaction may contain multiple line items.

Sales_Details → Product: Each line item refers to a specific product.

SalesHeader → TeeTimes: A transaction is linked to one tee time booking.

TeeTimes → CartAssignment → Carts: Each tee time includes a cart assignment, and each cart has its own identifier.

Normalization / Functional Dependencies:

The database design follows principles up to Third Normal Form (3NF):

1NF: All attributes are atomic (no multivalued fields).

2NF: Partial dependencies are eliminated by introducing associative entities (e.g., Sales_Details for line items).

3NF: All non-key attributes depend only on the primary key (e.g., in Product, Price depends on ProductID).

Overall, there are no more partial or transitive dependencies in any of the tables, so they are all in 3NF.

Data Dictionary:

Entity	Attribute	Description	Key Type
Customer	CustID	Unique customer ID	PK
	FName, LName	Customer name	-
	Address, City, State, Zip, Phone	Contact information	-
SalesHeader	SalesHeaderID	Transaction ID	PK
	CustID	The customer making the purchase	FK (to Customer)
	SalesDate, OrderTotal	Date and total value of transaction	-
Sales_Details	SalesDetailID	Unique ID for each line item	PK
	SalesHeaderID	Associated transaction	FK
	ProductID	Product sold	FK
	OrderQty, UnitPrice, LineTotal	Quantity and pricing details	-

Product	ProductID	Unique product ID	PK
	Prod_Name	Product name	-
	Prod_Type	Type (e.g., food, drink, apparel)	-
	Price	Base cost of the item	-
TeeTimes	TTID	Unique tee time ID	PK
	SalesHeaderID	Linked transaction	FK
	Date, StartTime	When the round is scheduled	-
	RoundType, Num_Players, StartHole	Round information	-
CartAssignment	CartAssignID	Unique cart assignment ID	PK
	TTID, CartID	Tee time and assigned cart	FK (both)
Carts	CartID	Unique cart ID	PK
	VIN, Date Purchased	Vehicle details	-

03 Implementation

To implement our database system, we began by identifying the primary business functions and entities relevant to a golf course operation. These included customers, sales transactions, tee time reservations, product purchases, group play, cart usage, and golf rounds. From these business functions, we derived a corresponding set of relational database tables: Customer, SalesHeader, SalesDetail, Product, TeeTimes, GolfGroup, GolfRound, Cart, and CartAssignment.

Our team collaborated by assigning each table to a specific group member based on interest and logical grouping of related tables. Each table was constructed using structured SQL CREATE TABLE statements with a focus on normalization, clear naming conventions, and the use of appropriate data types. We ensured that each table included a primary key for unique identification and implemented foreign key constraints to enforce referential integrity between related records. Check the appendix for the full list of table creation commands.

To support business processes such as calculating total order values or assigning customers to tee times, we also incorporated computed columns (e.g., LineTotal in Sales_Detail) and multi-column relationships. After defining the schema, we inserted realistic sample data to simulate actual business operations and applied UPDATE statements to reflect dynamic business changes (such as customer contact info updates or sales order adjustments). Look to the appendix for the full list of data entry commands.

This structured and collaborative approach resulted in a cohesive and functional database that accurately reflects the transactional and scheduling needs of a golf course business.

04 Operational SQL: Queries created and their business application

For the first business case, we focused on the revenues and profits from different products. For our first query (*Figure 4.0*), we found the most expensive product purchased by each customer to allow the manager to see an overview of the products with the highest margins that are selling. This nested query was created by combining the Customer table with the Sales Header, Order Details, and Products table. To piggyback off this query, we created a “Most Ordered Product” (*Figure 7.0*) table to further our insights into the products that are selling best to help manage inventory and ordering from our suppliers. After creating this query, we were able to get an accurate image of the most popular product, which led to the creation of the “Products that have not been purchased” (*Figure 10.0*). This table allows managers to get insight into which products are not selling and to halt orders from suppliers for that product. This included High Noons, Mom Waters, and HC Shot Glasses as well as more. This helps to get a grasp of what the customer is interested in, as well as helps the company save money on custom merchandise orders that are not creating revenue.

After exploring the most and least popular products, we looked to get a better understanding of the Havelka Clubs customers to best serve them. We started with an “analysis of all customer engagement and revenue from the highest spenders” (*Figure 9.0*) to get a basic overview of not only who our highest spending customer is, but also how many visits they have had, how many rounds they have played, when their last visit was, average spend, unique products purchased, carts used, and product categories purchased. All this in addition to a basic overview of the customer demographic. To better understand their preferences, we created a query of Customer Preferences Analysis (*Figure 12.0*). This query dove into which customer is purchasing which product category, with what frequency, the quantity of that product category, total spend, and what percent of the total spend it is. By looking at this data, we would be able to understand both the customer and their spending habits more. Furthermore, we looked into the customers who use the carts more than average (*Figure 5.0*) to create a customer profile and better accommodate them. We found that Joey and David are using the carts more than average, and would therefore allow us to give them perks or other benefits when visiting and playing a round. Next, we looked at the Profits made on Orders over \$100 to see the profit margins we are earning. This led us to discover that we are making a \$0 profit on almost all orders. This is a red flag that needs to be addressed and will be brought to the attention of upper management as well

as the suppliers to create more desirable profits. The last customer-centered query we developed was to see if we had any customers in the database that had yet to make a purchase (*Figure 13.0*). We found one customer, Jim Halpert, whom we would put effort into enticing to either book a tee time or purchase a product at the pro shop.

The last business case we explored was the geography of customers to get an overarching view of where our customers are coming from in order to not only guide marketing strategies, but also cater the offerings of the pro shop to better suit their preferences. We started by looking at the total sales by state (*Figure 6.0*) and found that the most popular state was Ohio by a vast majority, but after that was Pennsylvania which came as a surprise and we elected we would send surveys out to the out-of-state customers to get both an understanding of why they came to Havelka Clubs and what they would like to see offered. To further our inquiry and to know which states people are coming from outside Ohio to create our survey, we created a query for all states not in Ohio or Bordering States (*Figure 8.0*). This allowed us to see that Karen, Skip, and Marina all came from farther states and would be the target customers for the survey and investigation.

05 Conclusion

Throughout this project, we were able to create a functional database for Havelka Clubs Golf Course that includes all necessary elements and transactions for an operating golf course. This not only helped us to develop a deeper understanding of the workings of a database, but also how to create and maintain a functional database for a business. We created an ERD and, through thorough editing and updating, were able to encompass all functionalities of a golf course and pro shop in third normal form. Furthermore, we developed three separate business cases that both tested our database and allowed us to develop analysis into our customer base, giving insights that would immensely benefit the upper management of Havelka Clubs.

Appendix

Figure 1.0: ER Diagram

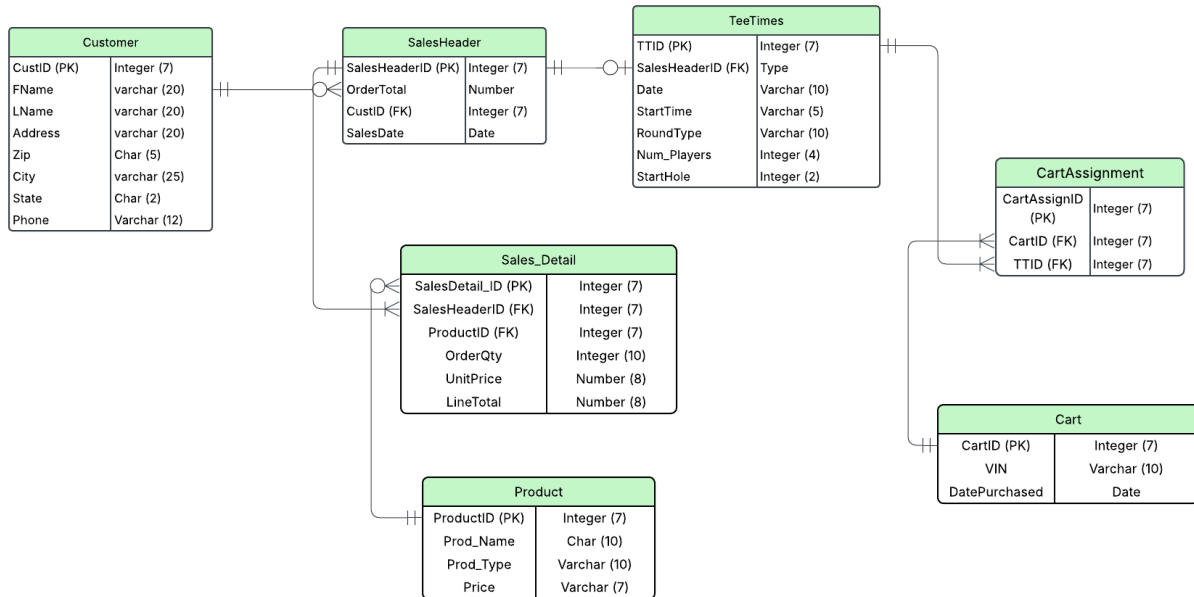


Figure 2.0: Table Creation and Alter Commands

Customer -

```

CREATE TABLE Customer
(CustID integer PRIMARY KEY,
Fname varchar(20),
Lname varchar(20),
Address varchar(20),
Zip char(5),
City varchar(25),
State char(2),
Phone char(10),
Email varchar(25));
    
```

SalesHeader -

```

CREATE TABLE SalesHeader
(SalesHeaderID integer PRIMARY KEY,
OrderTotal numeric(7,2),
SalesDate date,
CustID integer,
CONSTRAINT SalesHeader_FK FOREIGN KEY (CustID) REFERENCES Customer);
    
```

Sales_Detail -

```

CREATE TABLE Sales_Detail
    
```

```
(SalesDetail_ID integer,  
SalesHeaderID integer,  
ProductID integer,  
OrderQty integer,  
UnitPrice number(9, 2),  
LineTotal AS (OrderQty * UnitPrice),  
CONSTRAINT Sales_Detail_Sales_Detail_ID_pk PRIMARY KEY (SalesDetail_ID),  
CONSTRAINT Sales_Detail_SalesHeaderID_fk FOREIGN KEY (SalesHeaderID) REFERENCES  
SalesHeader(SalesHeaderID),  
CONSTRAINT Sales_Detail_ProductID_fk FOREIGN KEY (ProductID) REFERENCES  
Product(ProductID));
```

Product -

```
CREATE TABLE Product  
(ProductID integer,  
Prod_Name varchar(20),  
Prod_Type varchar(20),  
Price number(9, 2),  
CONSTRAINT Product_ProductID_pk PRIMARY KEY (ProductID));
```

TeeTimes -

```
CREATE TABLE TeeTimes  
( TTID NUMBER(7),  
SalesHeaderID NUMBER(7),  
TeeDate DATE,  
StartTime TIMESTAMP,  
RoundType VARCHAR2(10),  
Num_Players NUMBER(4),  
StartHole INTEGER(2),  
CONSTRAINT TeeTimes_TTID_pk PRIMARY KEY (TTID),  
CONSTRAINT SalesHeader_SalesHeaderID_fk FOREIGN KEY (SalesHeaderID) REFERENCES  
SalesHeader(SalesHeaderID));
```

Cart -

```
CREATE TABLE Cart  
(CartID INT PRIMARY KEY,  
VIN VARCHAR(10),  
Mileage VARCHAR(6));  
ALTER TABLE Cart  
DROP COLUMN Mileage  
ALTER TABLE Cart  
ADD (DatePurchased Date);
```

CartAssignment -

```
CREATE TABLE CartAssignment
( CartAssignmentID Integer Primary Key,
  CartID Integer,
  TTID Integer,
  CONSTRAINT CartAssignment_CartID_FK
  FOREIGN KEY (CartID) References Cart(CartID),
  CONSTRAINT CartAssignment_TTID_FK
  FOREIGN KEY (TTID) References TeeTimes(TTID));
```

Figure 3.0: Data Creation and Implementation Commands

Customer -

```
INSERT INTO Customer VALUES(10001, 'Joey', 'Endres', '8 Wooster Pl', '45056', 'Oxford', 'OH',
'6142597873', 'joe.endres.1216@gmail.com');
INSERT INTO Customer VALUES(10002, 'David', 'Taylor', '33 Homestead Rd', '17033', 'Hershey', 'PA',
'7172639983', 'david.taylor@gmail.com');
INSERT INTO Customer VALUES(10003, 'Isobel', 'Bartels', '729 Meadowood Way', '43614', 'Napoleon',
'OH', '8882793645', 'isobel.bartels@gmail.com');
INSERT INTO Customer VALUES(10004, 'Marina', 'Erceg', '09 Parkview Ave', '60007', 'Chicago', 'IL',
'312', 'marina.erceg@gmail.com');
INSERT INTO Customer VALUES(10005, 'Doug', 'Havelka', '328 S Spring St', '45056', 'Oxford', 'OH',
'5132843090', 'doug.havelka@gmail.com');
INSERT INTO Customer VALUES(10006, 'Charlie', 'Franks', '2 Airy Ln', '45201', 'Cincinnati', 'OH',
'5138934562', 'charlie.franks@gmail.com');
INSERT INTO Customer VALUES(10007, 'Skip', 'Bayless', 'Prairie View St', '20092', 'Bentonville', 'AR',
'8932208315', 'skip.bayless@gmail.com');
INSERT INTO Customer VALUES(10008, 'Julia', 'Michaels', '803 Mtn Pass Way', '43029', 'Columbus',
'OH', '6142893029', 'david.taylor@gmail.com');
INSERT INTO Customer VALUES(10009, 'Johnny', 'Rodrick', '218 Fermenton St', '43082', 'Galena',
'OH', '6142803743', 'johnny.rodrick@gmail.com');
INSERT INTO Customer VALUES(10010, 'Karen', 'Karen', 'West S Ave', '39437', 'Savanna', 'GA',
'9243789983', 'karen.karen@gmail.com');
INSERT INTO Customer VALUES(10011, 'Jim', 'Halpert', '117 Vonburgen St.', '18510', 'Scranton', 'PA',
'3987462237', 'jim.halpert@gmail.com');
```

```
UPDATE Customer
SET Phone = '3129893876'
WHERE CustID = 10004
```

```
UPDATE Customer
SET Email = 'julia.michaels@gmail.com'
WHERE CustID = 10008
```

SalesHeader -

```
INSERT INTO SalesHeader VALUES(101, 45.99, TO_DATE('2025-03-22', 'YYYY-MM-DD'),
100010);
INSERT INTO SalesHeader VALUES(102, 183.96, TO_DATE('2025-03-23', 'YYYY-MM-DD'),
10009);
INSERT INTO SalesHeader VALUES(103, 91.98, TO_DATE('2025-03-24', 'YYYY-MM-DD'), 10008);
INSERT INTO SalesHeader VALUES(104, 91.98, TO_DATE('2025-03-25', 'YYYY-MM-DD'), 10007);
INSERT INTO SalesHeader VALUES(105, 367.92, TO_DATE('2025-03-26', 'YYYY-MM-DD'),
10006);
INSERT INTO SalesHeader VALUES(106, 183.96, TO_DATE('2025-03-26', 'YYYY-MM-DD'),
10005);
INSERT INTO SalesHeader VALUES(107, 45.99, TO_DATE('2025-03-26', 'YYYY-MM-DD'), 10004);
INSERT INTO SalesHeader VALUES(108, 45.99, TO_DATE('2025-03-29', 'YYYY-MM-DD'), 10003);
INSERT INTO SalesHeader VALUES(109, 275.94, TO_DATE('2025-03-29', 'YYYY-MM-DD'),
10002);
INSERT INTO SalesHeader VALUES(1010, 45.99, TO_DATE('2025-03-30', 'YYYY-MM-DD'),
10001);
```

```
UPDATE SalesHeader
SET OrderTotal = 52.89
WHERE SalesHeaderID = 101;
UPDATE SalesHeader
SET OrderTotal = 52.50
WHERE SalesHeaderID = 102;
UPDATE SalesHeader
SET OrderTotal = 168
WHERE SalesHeaderID = 103;
UPDATE SalesHeader
SET OrderTotal = 47.98
WHERE SalesHeaderID = 104;
UPDATE SalesHeader
SET OrderTotal = 29900
WHERE SalesHeaderID = 105;
UPDATE SalesHeader
SET OrderTotal = 74.74
WHERE SalesHeaderID = 106;
UPDATE SalesHeader
SET OrderTotal = 10.50
WHERE SalesHeaderID = 107;
UPDATE SalesHeader
SET OrderTotal = 39.96
WHERE SalesHeaderID = 108;
UPDATE SalesHeader
SET OrderTotal = 204.21
WHERE SalesHeaderID = 109;
```

```
UPDATE SalesHeader
SET OrderTotal = 15.99
WHERE SalesHeaderID = 1010;
```

```
INSERT INTO SalesHeader VALUES(1011, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10001);
INSERT INTO SalesHeader VALUES(1012, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10002);
INSERT INTO SalesHeader VALUES(1013, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10003);
INSERT INTO SalesHeader VALUES(1014, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10004);
INSERT INTO SalesHeader VALUES(1015, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10005);
INSERT INTO SalesHeader VALUES(1016, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10006);
INSERT INTO SalesHeader VALUES(1017, 0, TO_DATE('2023-09-22', 'YYYY-MM-DD'), 10007);
INSERT INTO SalesHeader VALUES(1018, 0, TO_DATE('2023-09-23', 'YYYY-MM-DD'), 10001);
INSERT INTO SalesHeader VALUES(1019, 0, TO_DATE('2023-09-23', 'YYYY-MM-DD'), 10002);
```

```
UPDATE SalesHeader
SET OrderTotal = 183.96
WHERE SalesHeaderID = 1011;
UPDATE SalesHeader
SET OrderTotal = 137.97
WHERE SalesHeaderID = 1012;
UPDATE SalesHeader
SET OrderTotal = 45.98
WHERE SalesHeaderID = 1013;
UPDATE SalesHeader
SET OrderTotal = 183.96
WHERE SalesHeaderID = 1014;
UPDATE SalesHeader
SET OrderTotal = 183.96
WHERE SalesHeaderID = 1015;
UPDATE SalesHeader
SET OrderTotal = 137.97
WHERE SalesHeaderID = 1016;
UPDATE SalesHeader
SET OrderTotal = 22.99
WHERE SalesHeaderID = 1017;
UPDATE SalesHeader
SET OrderTotal = 183.96
WHERE SalesHeaderID = 1018;
UPDATE SalesHeader
SET OrderTotal = 45.98
WHERE SalesHeaderID = 1019;
```

SalesDetails -

```
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
```

```

VALUES(01, 102, 001, 43, 1.23);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(02, 102, 002, 10, 5.25);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(03, 103, 003, 24, 7.00);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(04, 104, 007, 2, 23.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(05, 105, 004, 10000, 2.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(06, 106, 004, 25, 2.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(07, 107, 008, 3, 3.50);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(08, 108, 010, 4, 9.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(09, 109, 011, 20, 2.25);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES(010, 1010, 020, 1, 15.99);

```

```

UPDATE Sales_Detail
SET OrderQty = 4
WHERE SalesDetail_ID = 12;

```

```

INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (13, 1011, 21, 4, 45.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (14, 1012, 21, 3, 45.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (15, 1013, 22, 2, 22.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (16, 1014, 21, 4, 45.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (17, 1015, 21, 4, 45.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (18, 1016, 21, 3, 45.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (19, 1017, 22, 1, 22.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (20, 1018, 21, 4, 45.99);
INSERT INTO Sales_Detail (SalesDetail_ID, SalesHeaderID, ProductID, OrderQty, UnitPrice)
VALUES (21, 1019, 22, 2, 22.99);

```

Product -


```

INSERT INTO Product VALUES('001', 'Coca-Cola', 'Beverage', '3.99');
INSERT INTO Product VALUES(002, 'Baja Blast', 'Beverage', 2.99);
INSERT INTO Product VALUES(003, 'Hot Chocolate', 'Beverage', 5.50);
INSERT INTO Product VALUES(004, 'Bapple', 'Beverage', 1.99);
INSERT INTO Product VALUES(005, 'High Noon', 'Beverage', 3.99);
INSERT INTO Product VALUES(006, 'Mom Water', 'Beverage', 10.90);
INSERT INTO Product VALUES(007, 'Golden Fingers', 'Food', 19.64);
INSERT INTO Product VALUES(008, 'Cheetos', 'Food', 2.50);
INSERT INTO Product VALUES(009, 'Sunchips', 'Food', 3.00);
INSERT INTO Product VALUES(010, 'Creeper Crunch', 'Food', 6.99);
INSERT INTO Product VALUES(011, 'Hersheys Chocolate', 'Food', 2.00);
INSERT INTO Product VALUES(012, 'Blink Blorks', 'Food', 0.01);
INSERT INTO Product VALUES(013, 'Gushers', 'Food', 1.25);
INSERT INTO Product VALUES(014, 'Trolli Gummy Worms', 'Food', 3.43);
INSERT INTO Product VALUES(015, 'HC Balls', 'Merch', 12.99);
INSERT INTO Product VALUES(016, 'HC Shotglass', 'Merch', 7.99);
INSERT INTO Product VALUES(017, 'HC T-shirt', 'Merch', 17.98);
INSERT INTO Product VALUES(018, 'HC Sweater', 'Merch', 25.99);
INSERT INTO Product VALUES(019, 'HC Block of Gold', 'Merch', 9999.99);
INSERT INTO Product VALUES(020, 'Club Cleaner', 'Misc', 13.99);
INSERT INTO Product VALUES(021, '18 Holes Game', 'Golf Round', 45.99);
INSERT INTO Product VALUES(022, '9 Holes Game', 'Golf Round', 22.99);

```

TeeTimes -

```

INSERT INTO TeeTimes VALUES (5001, 101, TO_DATE('2023-10-01', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-01 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 4);
INSERT INTO TeeTimes VALUES (5002, 102, TO_DATE('2023-10-01', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-01 08:15:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 3);
INSERT INTO TeeTimes VALUES (5003, 103, TO_DATE('2023-10-01', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-01 08:30:00', 'YYYY-MM-DD HH24:MI:SS'), '9-Hole', 2);
INSERT INTO TeeTimes VALUES (5004,104, TO_DATE('2023-10-01', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-01 08:45:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 4);
INSERT INTO TeeTimes VALUES (5005,105, TO_DATE('2023-10-02', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-02 09:00:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 4);
INSERT INTO TeeTimes VALUES (5006,106, TO_DATE('2023-10-02', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-02 09:15:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 3);
INSERT INTO TeeTimes VALUES (5007, 107, TO_DATE('2023-10-02', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-02 09:30:00', 'YYYY-MM-DD HH24:MI:SS'), '9-Hole', 1);
INSERT INTO TeeTimes VALUES (5008,108, TO_DATE('2023-10-03', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-03 10:00:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 4);
INSERT INTO TeeTimes VALUES (5009, 109, TO_DATE('2023-10-03', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-03 10:15:00', 'YYYY-MM-DD HH24:MI:SS'), '18-Hole', 4);
INSERT INTO TeeTimes VALUES (5010,1010, TO_DATE('2023-10-03', 'YYYY-MM-DD'),
    TO_TIMESTAMP('2023-10-03 10:30:00', 'YYYY-MM-DD HH24:MI:SS'), '9-Hole', 2);

```

```
UPDATE TeeTimes
SET SalesHeaderID = 1011
WHERE TTID = 5001;
UPDATE TeeTimes
SET SalesHeaderID = 1012
WHERE TTID = 5002;
UPDATE TeeTimes
SET SalesHeaderID = 1013
WHERE TTID = 5003;
UPDATE TeeTimes
SET SalesHeaderID = 1014
WHERE TTID = 5004;
UPDATE TeeTimes
SET SalesHeaderID = 1015
WHERE TTID = 5005;
UPDATE TeeTimes
SET SalesHeaderID = 1016
WHERE TTID = 5006;
UPDATE TeeTimes
SET SalesHeaderID = 1017
WHERE TTID = 5007;
UPDATE TeeTimes
SET SalesHeaderID = 1018
WHERE TTID = 5008;
UPDATE TeeTimes
SET SalesHeaderID = 1019
WHERE TTID = 5010;
```

GolfRound -

```
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (1, 6001, 1);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (2, 6002, 3);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (3, 6003, 5);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (4, 6004, 7);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (5, 6005, 9);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (6, 6006, 2);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (7, 6007, 4);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (8, 6008, 6);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (9, 6009, 8);
INSERT INTO GolfRound (RoundID, GroupID, StartHole) VALUES (10, 6010, 10);
```

Cart -

```
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (1, 'VIN0012345', '120.5');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (2, 'VIN0012346', '98.2');
```

```
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (3, 'VIN0012347', '110.0');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (4, 'VIN0012348', '85.7');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (5, 'VIN0012349', '102.3');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (6, 'VIN0012350', '75.4');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (7, 'VIN0012351', '90.0');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (8, 'VIN0012352', '88.8');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (9, 'VIN0012353', '100.1');
INSERT INTO Cart (CartID, VIN, Mileage) VALUES (10, 'VIN0012354', '95.6');
INSERT INTO Cart VALUES(11, 'VIN0012355', '203.2');
INSERT INTO Cart VALUES(12, 'VIN0012356', '83.3');
INSERT INTO Cart VALUES(13, 'VIN0012357', '163.0');
INSERT INTO Cart VALUES(14, 'VIN0012358', '309.4');
INSERT INTO Cart VALUES(15, 'VIN0012359', '114.2');
INSERT INTO Cart VALUES(16, 'VIN0012360', '125.8');
```

```
UPDATE Cart
SET DatePurchased = TO_DATE('2022-12-09', 'YYYY-MM-DD')
WHERE CartID = 1;
UPDATE Cart
SET DatePurchased = TO_DATE('2022-12-09', 'YYYY-MM-DD')
WHERE CartID = 2;
UPDATE Cart
SET DatePurchased = TO_DATE('2022-12-09', 'YYYY-MM-DD')
WHERE CartID = 3;
UPDATE Cart
SET DatePurchased = TO_DATE('2022-12-10', 'YYYY-MM-DD')
WHERE CartID = 4;
UPDATE Cart
SET DatePurchased = TO_DATE('2022-12-10', 'YYYY-MM-DD')
WHERE CartID = 5;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-03-11', 'YYYY-MM-DD')
WHERE CartID = 6;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-03-11', 'YYYY-MM-DD')
WHERE CartID = 7;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-04-27', 'YYYY-MM-DD')
WHERE CartID = 8;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-04-27', 'YYYY-MM-DD')
WHERE CartID = 9;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-04-27', 'YYYY-MM-DD')
```

```

WHERE CartID = 10;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-06-11', 'YYYY-MM-DD')
WHERE CartID = 11;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-06-11', 'YYYY-MM-DD')
WHERE CartID = 12;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-06-11', 'YYYY-MM-DD')
WHERE CartID = 13;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-08-23', 'YYYY-MM-DD')
WHERE CartID = 14;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-09-02', 'YYYY-MM-DD')
WHERE CartID = 15;
UPDATE Cart
SET DatePurchased = TO_DATE('2023-09-19', 'YYYY-MM-DD')
WHERE CartID = 16;

```

Cart Assignment -

```

INSERT INTO CartAssignment VALUES(301, 1, 5001);
INSERT INTO CartAssignment VALUES(302, 2, 5001);
INSERT INTO CartAssignment VALUES(303, 3, 5002);
INSERT INTO CartAssignment VALUES(304, 4, 5002);
INSERT INTO CartAssignment VALUES(305, 5, 5003);
INSERT INTO CartAssignment VALUES(306, 6, 5004);
INSERT INTO CartAssignment VALUES(307, 7, 5004);
INSERT INTO CartAssignment VALUES(308, 8, 5005);
INSERT INTO CartAssignment VALUES(309, 9, 5005);
INSERT INTO CartAssignment VALUES(310, 10, 5006);
INSERT INTO CartAssignment VALUES(311, 11, 5006);
INSERT INTO CartAssignment VALUES(312, 12, 5007);
INSERT INTO CartAssignment VALUES(313, 1, 5008);
INSERT INTO CartAssignment VALUES(314, 2, 5008);
INSERT INTO CartAssignment VALUES(315, 3, 5009);
INSERT INTO CartAssignment VALUES(316, 4, 5009);
INSERT INTO CartAssignment VALUES(317, 5, 5010);

```

Figure 4.0: Operational SQL Query 1

Finding the most expensive product purchased by each customer

```

SELECT CustID, Fname, Lname, Prod_Name, Price
FROM Customer
JOIN SalesHeader USING (CustID)

```

```

JOIN Sales_Detail USING (SalesHeaderID)
JOIN Product USING (ProductID)
WHERE (SalesHeaderID, Price) IN (
    SELECT SalesHeaderID, MAX(Price)
    FROM Sales_Detail
    JOIN Product USING (ProductID)
    GROUP BY SalesHeaderID);

```

CUSTID	FNAME	LNAME	PROD_NAME	PRICE
10002	David	Taylor	18 Holes Game	45.99
10010	Karen	Karen	Coca Cola	3.99
10009	Johnny	Rodrick	Baja Blast	2.99
10008	Julia	Michaels	Hot Chocolate	5.5
10007	Skip	Bayless	Golden Fingers	19.64
10006	Charlie	Franks	Bapple	1.99
10005	Doug	Havelka	Bapple	1.99
10004	Marina	Erceg	Cheetos	2.5
10003	Isobel	Bartels	Creeper Crunch	6.99
10001	Joey	Endres	Club Cleaner	13.99
10001	Joey	Endres	18 Holes Game	45.99
10002	David	Taylor	18 Holes Game	45.99
10003	Isobel	Bartels	9 Holes Game	22.99
10004	Marina	Erceg	18 Holes Game	45.99
10005	Doug	Havelka	18 Holes Game	45.99
10006	Charlie	Franks	18 Holes Game	45.99
10007	Skip	Bayless	9 Holes Game	22.99
10001	Joey	Endres	18 Holes Game	45.99
10002	David	Taylor	9 Holes Game	22.99

Figure 5.0: Operational SQL Query 2

Finding customers who have used more carts than the average

```

SELECT CustID, Fname, Lname, COUNT(CartID) AS cart_count
FROM Customer
JOIN SalesHeader USING (CustID)
JOIN TeeTimes USING(SalesHeaderID)
JOIN CartAssignment USING(TTID)
GROUP BY CustID, Fname, Lname
HAVING COUNT(CartID) > (
    SELECT AVG(cart_per_customer)
    FROM (
        SELECT COUNT(CartID) AS cart_per_customer
        FROM Customer
        JOIN SalesHeader USING (CustID)
        JOIN TeeTimes USING(SalesHeaderID)
        JOIN CartAssignment USING (TTID)
        GROUP BY CustID));

```

SELECT c.CustID, c.Fname, c.Lname, COUNT(ca.CartID) AS cart_count
FROM Customer c
JOIN SalesHeader sh ON c.CustID = sh.CustID
JOIN TeeTimes tt ON sh.SalesHeaderID = tt.SalesHeaderID
JOIN CartAssignment ca ON tt.TTID = ca.TTID
GROUP BY c.CustID, c.Fname, c.Lname
HAVING COUNT(ca.CartID) > (
SELECT AVG(cart_per_customer)
FROM (
SELECT COUNT(ca2.CartID) AS cart_per_customer

Last 15 SQL state
SELECT c.CustID

Execute Query
Save Script
Clear

Query returned: 2 row(s)

Download CSV

CUSTID	FNAME	LNAME	CART_COUNT
10001	Joey	Endres	4
10002	David	Taylor	5

Figure 6.0: Operational SQL Query 3

Total Sales by State

```

SELECT
    Customer.State,
    SUM(SalesHeader.OrderTotal) AS TotalSales
FROM Customer
JOIN SalesHeader USING (CustID)
GROUP BY Customer.State
ORDER BY TotalSales DESC;

```

STATE	TOTALSALES
OH	30987.02
PA	388.16
IL	194.46
AR	70.97
GA	52.89

Figure 7.0: Operational SQL Query 4

Most Ordered Product;

```

SELECT Prod_Name, SUM(OrderQty) AS TotalOrdered
FROM Product
JOIN Sales_Detail USING (ProductID)
GROUP BY Prod_Name
ORDER BY TotalOrdered DESC

```

FETCH FIRST 5 ROWS ONLY;

PROD_NAME	TOTALORDERED
Bapple	10025
Coca Cola	43
18 Holes Game	26
Hot Chocolate	24
Hersheys Chocolate	20
Baja Blast	10
9 Holes Game	5
Creeper Crunch	4
Cheetos	3
Golden Fingers	2

Figure 8.0: Operational SQL Query 5

All States Not in Ohio or Bordering States

```
SELECT UNIQUE Fname, Lname, state
FROM Customer
WHERE NOT STATE IN('PA', 'OH')
ORDER BY State
```

```
SELECT UNIQUE Fname, Lname, state
FROM Customer
WHERE NOT STATE IN('PA', 'OH')
ORDER BY State
```

Last 15 SQL state
SELECT UNIQUE

Execute Query

Save Script

Clear

Query returned: 3 row(s)

Download CSV

FNAME	LNAME	STATE
Skip	Bayless	AR
Karen	Karen	GA
Marina	Erceg	IL

Figure 9.0: Operational SQL Query 6

Analysis of all Customer Engagement and Revenue from highest spenders

```
SELECT CustID, Fname || ' ' || Lname AS CustomerName, Email, City, State,
COUNT(DISTINCT SalesHeaderID) AS TotalVisits, COUNT(DISTINCT
TTID) AS GolfRoundsPlayed, SUM(OrderTotal) AS TotalSpent,
MAX(SalesDate) AS LastVisitDate, ROUND(AVG(OrderTotal), 2) AS
```

```

AvgSpendPerVisit, COUNT(DISTINCT ProductID) AS
UniqueProductsPurchased, COUNT(DISTINCT CartID) AS CartsUsed,
LISTAGG(DISTINCT Prod_Type, ', ') WITHIN GROUP (ORDER BY
Prod_Type) AS ProductCategoriesPurchased
FROM Customer
LEFT OUTER JOIN SalesHeader USING (CustID)
LEFT OUTER JOIN TeeTimes USING (SalesHeaderID)
LEFT OUTER JOIN Sales_Detail USING (SalesHeaderID)
LEFT OUTER JOIN Product USING (ProductID)
LEFT OUTER JOIN CartAssignment USING (TTID)
GROUP BY CustID, Fname, Lname, Email, City, State
ORDER BY TotalSpent DESC NULLS LAST;

```

CUSTID	CUSTOMERNAME	EMAIL	CITY	STATE	TOTALVISITS	GOLFGROUNDSPLAYED	TOTALSPENT	LASTVISITDATE	AVGSPENDPERVISIT	UNIQUEPRODUCTSPURCHASED	CARTSUSED	PRODUCTCATEGORIESPURCHASED
10006	Charlie Franks	charlie.franks@gmail.com	Cincinnati	OH	2	1	30175.94	26-Mar-2025	10058.65	2	2	Beverage, Golf Round
10002	David Taylor	david.taylor@gmail.com	Hershey	PA	3	3	1547.18	29-Mar-2025	171.91	4	3	Food, Golf Round
10001	Joey Endres	joe.endres.1216@gmail.com	Oxford	OH	3	2	751.83	30-Mar-2025	150.37	2	2	Golf Round, Misc
10005	Doug Havelka	doug.havelka@gmail.com	Oxford	OH	2	1	442.66	26-Mar-2025	147.55	2	2	Beverage, Golf Round
10004	Marina Erceg	marina.erceg@gmail.com	Chicago	IL	2	1	378.42	26-Mar-2025	126.14	2	2	Food, Golf Round
10008	Julia Michaels	julia.michaels@gmail.com	Columbus	OH	1	0	168	24-Mar-2025	168	1	0	Beverage
10003	Isabel Bartels	isabel.bartels@gmail.com	Napoleon	OH	2	1	85.94	29-Mar-2025	42.97	2	1	Food, Golf Round
10007	Skip Bayless	skip.bayless@gmail.com	Bevensville	AR	2	1	70.97	25-Mar-2025	35.49	2	1	Food, Golf Round
10010	Karen Karen	karen.karen@gmail.com	Savanna	GA	1	0	52.89	22-Mar-2025	52.89	1	0	Beverage
10009	Johnny Rodrick	johnny.rodrick@gmail.com	Galena	OH	1	0	52.5	23-Mar-2025	52.5	1	0	Beverage

Figure 10.0: Operational SQL Query 7

Products That Haven't Been Purchased

```

SELECT ProductID, Prod_Name, Prod_Type, Price
FROM Product
LEFT JOIN Sales_Detail ON Product.ProductID = Sales_Detail.ProductID
WHERE Sales_Detail.ProductID IS NULL
ORDER BY ProductID;

```



```

SELECT
    Product.ProductID,
    Product.Prod_Name,
    Product.Prod_Type,
    Product.Price
FROM Product
LEFT JOIN Sales_Detail ON Product.ProductID = Sales_Detail.ProductID
WHERE Sales_Detail.ProductID IS NULL
ORDER BY ProductID;

```

Execute Query

Save Script

Clear

Last 15 SQL statements

SELECT Product.ProductID, Produc

Query returned: 10 row(s)

Download CSV

1 to 10 ▾

PRODUCTID	PROD_NAME	PROD_TYPE	PRICE
5	High Noon	Beverage	3.99
6	Mom Water	Beverage	10.9
9	Sunchips	Food	3
12	Blink Borks	Food	.01
13	Gushers	Food	1.25
15	HC Balls	Merch	12.99
16	HC Shotglass	Merch	7.99
17	HC T-shirt	Merch	17.98
18	HC Sweater	Merch	25.99
19	HC Block of Gold	Merch	9999.99

Figure 11.0: Operational SQL Query 8

Profit Made on Orders More Than 100 Dollars

```

SELECT Fname, Lname, Ordertotal, Linetotal, (Ordertotal - Linetotal) AS "Profit"
FROM Customer
JOIN SalesHeader USING (CustID)
JOIN Sales_Detail USING (SalesHeaderID)
JOIN Product USING (ProductID)
WHERE Linetotal > 100

```

SELECT Fname, Lname, Ordertotal, Linetotal, (Ordertotal - Linetotal) AS "Profit" FROM Customer JOIN SalesHeader USING (CustID) JOIN Sales_Detail USING (SalesHeaderID) JOIN Product USING (ProductID) WHERE Linetotal > 100

Last 15 SQL statements
SELECT Fname, Lname, Ordertotal

Execute Query
Save Script
Clear

Query returned: 9 row(s)

Download CSV
1 to 9 ▼

FNAME	LNAME	ORDERTOTAL	LINETOTAL	Profit
David	Taylor	204.21	183.96	20.25
Julia	Michaels	168	168	0
Charlie	Franks	29900	29900	0
Joey	Endres	183.96	183.96	0
David	Taylor	137.97	137.97	0
Marina	Erceg	183.96	183.96	0
Doug	Havelka	183.96	183.96	0
Charlie	Franks	137.97	137.97	0
Joey	Endres	183.96	183.96	0

Figure 12.0: Operational SQL Query 9

Customer Preferences Analysis

```

SELECT CustID, Fname || ' ' || Lname AS CustomerName, Prod_Type AS
ProductCategory, COUNT(DISTINCT SalesDetail_ID) AS PurchaseCount,
SUM(OrderQty) AS TotalQuantity, SUM(LineTotal) AS TotalSpend,
ROUND(SUM(LineTotal) * 100 / (SELECT SUM(LineTotal)
FROM Sales_Detail
JOIN SalesHeader USING(SalesHeaderID)
WHERE CustID = CustID), 2) AS PercentOfTotalSpend
FROM Customer
JOIN SalesHeader USING (CustID)
JOIN Sales_Detail USING( SalesHeaderID)
JOIN Product USING (ProductID)
GROUP BY CustID, Fname, Lname, Prod_Type
ORDER BY CustID, TotalSpend DESC;

```

CUSTID	CUSTOMERNAME	PRODUCTCATEGORY	PURCHASECOUNT	TOTALQUANTITY	TOTALSPEND	PERCENTOFTOTALSPEND
10001	Joey Endres	Golf Round	2	8	367.92	95.83
10001	Joey Endres	Misc	1	1	15.99	4.17
10002	David Taylor	Golf Round	3	9	367.91	84.94
10002	David Taylor	Food	2	21	65.25	15.06
10003	Isobel Bartels	Golf Round	1	2	45.98	53.5
10003	Isobel Bartels	Food	1	4	39.96	46.5
10004	Marina Erceg	Golf Round	1	4	183.96	94.6
10004	Marina Erceg	Food	1	3	10.5	5.4
10005	Doug Havelka	Golf Round	1	4	183.96	71.11
10005	Doug Havelka	Beverage	1	25	74.75	28.89
10006	Charlie Franks	Beverage	1	10000	29900	99.54
10006	Charlie Franks	Golf Round	1	3	137.97	.46
10007	Skip Bayless	Food	1	2	47.98	67.61
10007	Skip Bayless	Golf Round	1	1	22.99	32.39
10008	Julia Michaels	Beverage	1	24	168	100
10009	Johnny Rodrick	Beverage	1	10	52.5	100
10010	Karen Karen	Beverage	1	43	52.89	100

Figure 13.0: Operational SQL Query 10

Customers who are in the system but have not bought anything

SELECT SalesHeader.CustID, Fname, Lname, Email, Phone
FROM Customer
LEFT JOIN SalesHeader ON Customer.CustID = SalesHeader.CustID
WHERE SalesHeader.CustID IS NULL;

SELECT SalesHeader.CustID, Fname, Lname, Email, Phone

FROM Customer

LEFT JOIN SalesHeader ON Customer.CustID = SalesHeader.CustID

WHERE SalesHeader.CustID IS NULL;

Last 15 SQL statements

SELECT SalesHeader.CustID, Fname,

Execute Query

Save Script

Clear

Query returned: 1 row(s)

Download CSV

1 to 1 ▾

CUSTID	FNAME	LNAME	EMAIL	PHONE
(null)	Jim	Halpert	jim.halpert@gmail.com	3987462237